

CSc 8830 Assignment 6: Motion Tracking and Structure from Motion

Raghuveer Gummadi

Live app: <https://ragman95.pythonanywhere.com/> (Module 6: <https://ragman95.pythonanywhere.com/module-6/>)

Code: https://github.com/RagDoll95/CSC8830_Assignments/tree/main/week_6/module6

The app runs the repository scripts: `motion.py` (flow video), `validate.py` (two-frame tracking check and flow statistics) and `sfm.py` (four-view factorization). Theory and method follow the course lectures [N1] *Optical Flow* (FPCV-4-3), [N2] *Structure from Motion* (FPCV-4-4) and [N3] *Object Tracking* (FPCV-5-1); slide numbers are given as, e.g., [N1, s.17].

1 Optical flow and motion tracking

Videos. Penguin Walk, 00:30–01:00 [V1], and Sand Marble Race, 02:00–02:30 [V2]: 30 s each, 640×360 , 25 and 50 frames/s. In every frame up to 100 corners are tracked into the next frame with coarse-to-fine Lucas–Kanade [N1, s.26–36] (OpenCV’s implementation of [Bouguet 2000], 21×21 window, 4 pyramid levels). The arrows form the flow video; all vectors are in `points.csv`.

1.1 What optical flow tells us, with evidence

Optical flow is the motion of brightness patterns in the image; ideally it equals the *motion field*, the projection of 3-D scene motion [N1, s.3–6]. Each vector’s length is the point’s image speed and its direction is the direction of motion [N1, s.6]. From our videos:

1. **Object motion: speed and direction.** The white marble moves $(-2.0, -3.8)$ px/frame (≈ 214 px/s, up-left) and the yellow one $(-0.4, +1.0)$ px/frame. Without the 3-D geometry of the track these stay in pixels; with a known ground plane they could be converted to real speeds, as in traffic monitoring [N1, s.44].
2. **Camera motion (dominant flow).** The flow shared by most points is the background, i.e. the camera’s motion; removing it stabilizes video [N1, s.46]. Taking each frame’s median vector as that dominant flow: the hand-held penguin camera moves by a median 3.8 px/frame (only 1% of vectors are static), the marble camera pans smoothly at 2.7 px/frame, about $(-4.3, 1.7)$ in the first pair (plots below).
3. **Independently moving objects.** Vectors that disagree with the dominant flow (magenta) mark things moving on their own: in the penguin pair they lie on the penguins and the keeper’s legs and hands, while the railing and boards follow the camera (cyan).
4. **Relative depth.** For a translating camera, nearer points move faster [N1, s.6]. In the marble race all sand moves the same way, but the median speed falls from 5.3 (left third) to 3.9 (middle) to 2.9 px/frame (right third) over the first 4 s: consistent with the sand being nearer on the left. (Camera rotation can also make flow non-uniform, so this is relative, not metric, depth.)
5. **Where flow differs from the motion field, and where it fails.** The race’s scoreboard and timer overlays give zero flow (31% of vectors are static) although the camera moves: flow measures brightness motion, not scene motion [N1, s.7–8]. Spikes in the plots are scene cuts (marbles 19.6 s, penguins 24.1 s) and a fast pan (penguins 7.2–7.5 s, over 30 px/frame), where the small-motion assumption breaks [N1, s.34].

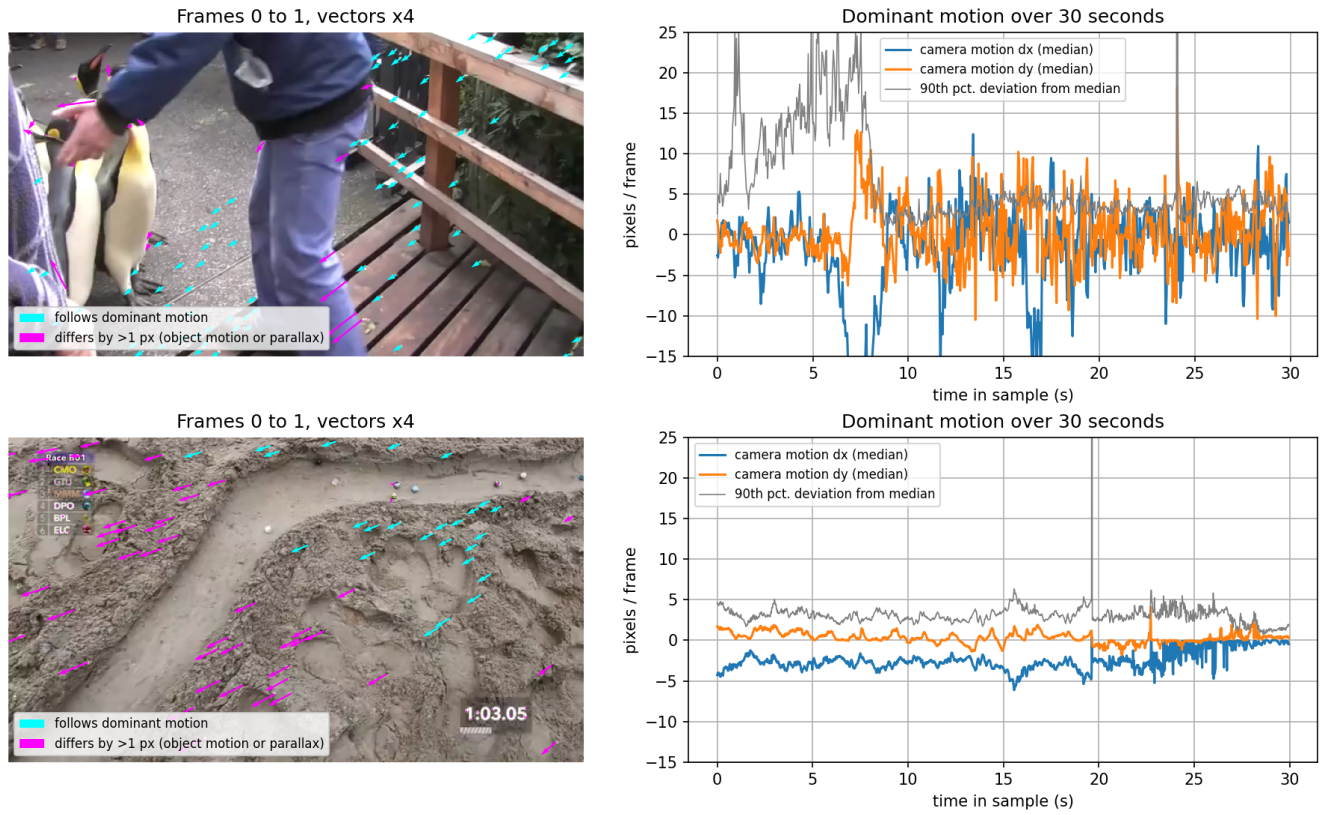


Figure 1: Left: frame 0→1 flow (vectors $\times 4$); cyan follows the frame's median (dominant) flow, magenta differs by more than 1 px. Right: dominant flow over the 30s and the 90th percentile of deviation from it. Top: penguins; bottom: marbles.

1.2 Tracking equations from first principles

The tracking problem for two frames. Given the location of a target (a point and a small window W around it) in the frame at time t , find its location in the frame at $t + \delta t$ [N3, s.22]. Template matching answers this by minimizing the sum of squared differences over a shift $(\delta x, \delta y)$ [N3, s.25; N1, s.40]:

$$\text{SSD}(\delta x, \delta y) = \sum_{(x,y) \in W} [I(x + \delta x, y + \delta y, t + \delta t) - I(x, y, t)]^2.$$

This is slow over a large search and limited to whole pixels. The optical flow equations give the minimizer directly, at sub-pixel accuracy.

Assumption 1, brightness constancy [N1, s.13]: $I(x + \delta x, y + \delta y, t + \delta t) = I(x, y, t)$.

Assumption 2, small motion [N1, s.14–16]. The first-order Taylor expansion is

$$I(x + \delta x, y + \delta y, t + \delta t) \approx I(x, y, t) + I_x \delta x + I_y \delta y + I_t \delta t.$$

Subtracting the two, dividing by δt and letting $\delta t \rightarrow 0$ gives the **optical flow constraint equation** [N1, s.17]

$$I_x u + I_y v + I_t = 0, \quad (u, v) = \left(\frac{dx}{dt}, \frac{dy}{dt} \right).$$

I_x, I_y, I_t come from two frames with the $2 \times 2 \times 2$ pixel cube [N1, s.18], e.g. $I_x(i, j, t) = \frac{1}{4} [I(i+1, j, t) + I(i+1, j, t+1) + I(i+1, j+1, t) + I(i+1, j+1, t+1)] - \frac{1}{4} [I(i, j, t) + I(i, j, t+1) + I(i, j+1, t) + I(i, j+1, t+1)]$. One equation in two unknowns only fixes the *normal flow*, $|u_n| = |I_t| / \sqrt{I_x^2 + I_y^2}$ along (I_x, I_y) : the aperture problem [N1, s.19–24].

Lucas–Kanade [N1, s.26–29]. Assume (u, v) is constant in the $n \times n$ window W . Each pixel $(k, l) \in W$ gives

$I_x(k, l)u + I_y(k, l)v = -I_t(k, l)$, so

$$\underbrace{\begin{bmatrix} I_x(1, 1) & I_y(1, 1) \\ \vdots & \vdots \\ I_x(n, n) & I_y(n, n) \end{bmatrix}}_{A \ (n^2 \times 2)} \underbrace{\begin{bmatrix} u \\ v \end{bmatrix}}_{\mathbf{u}} = \underbrace{\begin{bmatrix} -I_t(1, 1) \\ \vdots \\ -I_t(n, n) \end{bmatrix}}_{B \ (n^2 \times 1)}, \quad \underbrace{\begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix}}_{A^T A} \begin{bmatrix} u \\ v \end{bmatrix} = \underbrace{\begin{bmatrix} -\sum I_x I_t \\ -\sum I_y I_t \end{bmatrix}}_{A^T B},$$

$\mathbf{u} = (A^T A)^{-1} A^T B$. This is exactly the minimum of the linearized SSD above, $\sum_W (I_x u + I_y v + I_t)^2$. It is reliable only when $A^T A$ is well conditioned: both eigenvalues large and neither much larger than the other, i.e. a textured patch, not a smooth region or a single edge [N1, s.29–32]. The tracked point is $p_1 = p_0 + \mathbf{u}$.

Large motion [N1, s.34–36]. When the motion is more than about a pixel, the Taylor step fails. Build a pyramid by averaging 2×2 blocks (motion halves at each level), solve at the coarsest level, then at each finer level warp the earlier frame by the current flow, solve for the small remaining $(\Delta u, \Delta v)$ and add it.

1.3 Bilinear interpolation

The warp moves pixels to non-integer positions, so intensities must be sampled between pixel centres. Let $x = m + \alpha$, $y = n + \beta$ with $m = \lfloor x \rfloor$, $n = \lfloor y \rfloor$, $0 \leq \alpha, \beta < 1$. Interpolate linearly along x in the two neighbouring rows, $I(x, n) = (1 - \alpha)I_{m,n} + \alpha I_{m+1,n}$ and $I(x, n+1) = (1 - \alpha)I_{m,n+1} + \alpha I_{m+1,n+1}$, then along y :

$$I(x, y) = (1 - \beta)I(x, n) + \beta I(x, n+1) = (1 - \alpha)(1 - \beta)I_{m,n} + \alpha(1 - \beta)I_{m+1,n} + (1 - \alpha)\beta I_{m,n+1} + \alpha\beta I_{m+1,n+1}.$$

Each weight is the area of the opposite sub-rectangle; the four sum to 1, and the result is exact for any function $a + bx + cy + dxy$. Example: neighbours $\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}$, $\alpha = 0.25$, $\beta = 0.5$: rows 12.5 and 32.5, so $I = 22.5$.

1.4 Validation against actual pixel locations

For the first pair of consecutive frames in each video I marked recognizable points in frame 0 and found them again in frame 1 by eye on enlarged images (integer pixels, about ± 1 px; `measurements.csv`). These are compared with the Lucas–Kanade prediction and with SSD template matching (21×21 template, ± 10 px search).

Video	Point	p_0 (frame 0)	Measured p_1	LK $\hat{p}_1$	LK error	SSD match	SSD error
Penguins	Neck patch	(79, 49)	(78, 50)	(78.23, 49.96)	0.24	(78, 50)	0.00
Penguins	Beak tip	(114, 14)	(115, 12)	(111.63, 13.85)	3.84	(111, 14)	4.47
Marbles	White marble	(288, 102)	(286, 99)	(285.98, 98.25)	0.75	(286, 98)	1.00
Marbles	Blue marble	(432, 53)	(431, 51)	(430.79, 52.68)	1.69	(431, 52)	1.00
Marbles	Yellow marble	(428, 66)	(427, 67)	(427.60, 66.99)	0.60	(428, 67)	1.00

Worked solve (penguin neck). With cube derivatives over the $21 \times 21 = 441$ -pixel window at (79, 49):

$$\begin{bmatrix} 130018.6 & -20690.0 \\ -20690.0 & 47630.9 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} -119434.5 \\ 57979.9 \end{bmatrix}, \quad \det(A^T A) = 5.765 \times 10^9,$$

$$\begin{bmatrix} u \\ v \end{bmatrix} = \frac{1}{\det} \begin{bmatrix} 47630.9 & 20690.0 \\ 20690.0 & 130018.6 \end{bmatrix} \begin{bmatrix} -119434.5 \\ 57979.9 \end{bmatrix} = \begin{bmatrix} -0.779 \\ 0.879 \end{bmatrix} \text{ px/frame},$$

so one step predicts (78.22, 49.88), 0.25 px from the measured (78, 50). The eigenvalues of $A^T A$ are 4.3×10^4 and 1.3×10^5 : a well-conditioned, textured patch. At the beak tip they are only 2.3×10^3 and 8.1×10^3 (blurred, low contrast), and both LK and SSD miss by about 4 px. For the white marble one linear step gives $(-0.68, -1.24)$ and misses by 2.2 px, because the ≈ 4 px motion breaks the small-motion assumption; coarse-to-fine iteration reduces this to 0.75 px. All five systems are in `tracking_calculations.json`.

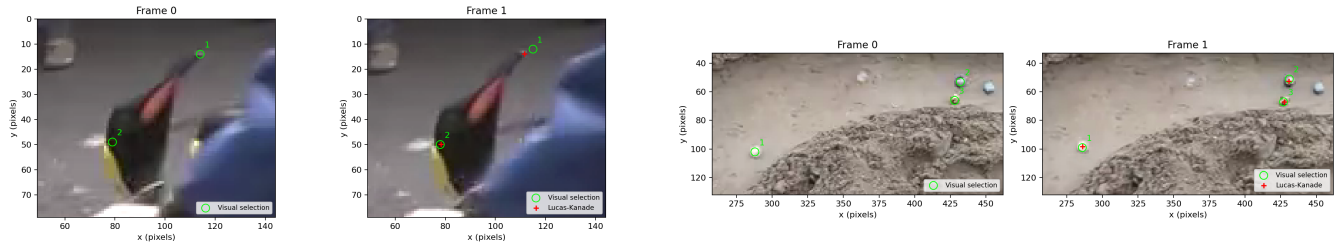


Figure 2: Measured points (green circles) and Lucas-Kanade predictions (red +) in frames 0 and 1. Left: penguins; right: marbles.

2 Structure from motion with four views of a planar object

Object, images and camera. The object is a flat 9×6 -corner checkerboard (24.9 mm squares) shown on a laptop screen, photographed from four directions with an iPhone (1× main lens, 3024×4032 portrait). Its 54 corners are detected automatically and are the tracked features [N2, s.5]. The camera parameters come from my Module 2 calibration (RMS reprojection 1.86 px):

$$K = \begin{bmatrix} 3336.69 & 0 & 1496.68 \\ 0 & 3335.58 & 2020.27 \\ 0 & 0 & 1 \end{bmatrix}, \quad (k_1, k_2, p_1, p_2, k_3) = (0.223, -1.376, 0.0006, -0.0008, 2.709).$$

They are used only to remove lens distortion and convert pixels to normalized image coordinates $K^{-1}[u, v, 1]^T$. The camera positions below come from the same calibration (the board's pose in each photo). The factorization does *not* use them; they are used only to check it. Tomasi-Kanade assumes one constant magnification [N2, s.8], so the four photos were chosen at nearly the same distance (1035–1072 mm) and from different directions.

View	Photo	Camera position (mm, board frame)	Distance	Tilt from normal	Depth range on board
1	IMG_3930	(44, -389, -930)	1035 mm	26.0°	7.9%
2	IMG_3917	(-419, -125, -915)	1069 mm	31.1°	11.5%
3	IMG_3928	(631, -502, -709)	1050 mm	47.5°	16.2%
4	IMG_3938	(-374, -538, -751)	1072 mm	45.5°	13.9%



Figure 3: The four views. Green: detected corners; red: reprojection of the factorization MS ; yellow: estimated boundary.

2.1 Method (Tomasi–Kanade factorization [N2])

Orthographic projection [N2, s.8–11]. Frame f has image axes $\mathbf{i}_f, \mathbf{j}_f$ (unit, orthogonal) and position C_f ; scene point P_p projects to $u_{f,p} = \mathbf{i}_f^T(P_p - C_f)$, $v_{f,p} = \mathbf{j}_f^T(P_p - C_f)$.

Centering trick [N2, s.12–14]. Put the world origin at the centroid of the points. Then the image centroid is $\bar{u}_f = -\mathbf{i}_f^T C_f$, and the centroid-subtracted coordinates $\tilde{u}_{f,p} = u_{f,p} - \bar{u}_f = \mathbf{i}_f^T P_p$, $\tilde{v}_{f,p} = \mathbf{j}_f^T P_p$ no longer contain C_f . Stacking all $F = 4$ frames and $N = 54$ points:

$$\underbrace{\begin{bmatrix} \tilde{u}_{1,1} & \cdots & \tilde{u}_{1,N} \\ \vdots & & \vdots \\ \tilde{u}_{F,1} & \cdots & \tilde{u}_{F,N} \\ \tilde{v}_{1,1} & \cdots & \tilde{v}_{1,N} \\ \vdots & & \vdots \\ \tilde{v}_{F,1} & \cdots & \tilde{v}_{F,N} \end{bmatrix}}_{W \ (2F \times N) \text{ known}} = \underbrace{\begin{bmatrix} \mathbf{i}_1^T \\ \vdots \\ \mathbf{i}_F^T \\ \mathbf{j}_1^T \\ \vdots \\ \mathbf{j}_F^T \end{bmatrix}}_{M \ (2F \times 3)} \underbrace{\begin{bmatrix} P_1 & \cdots & P_N \end{bmatrix}}_{S \ (3 \times N)}.$$

Rank [N2, s.24–27]. $\text{rank}(W) = \text{rank}(MS) \leq \min(\text{rank } M, \text{rank } S) \leq 3$. For a *planar* object, take the world z axis along the plane normal, so every $P_p = (x_p, y_p, 0)$ and $\text{rank}(S) = 2$. Then $\text{rank}(W) \leq 2$, and only the in-plane parts $\tilde{\mathbf{i}}_f, \tilde{\mathbf{j}}_f$ (first two components) of the camera axes appear in W : $W = \tilde{M}\tilde{S}$ with $\tilde{M}$ $2F \times 2$ and $\tilde{S}$ $2 \times N$.

SVD and factorization [N2, s.28–33]. $W = U\Sigma V^T$. Keep the two largest singular values: $W = U_1\Sigma_1 V_1^T$, and set $\tilde{M} = U_1\Sigma_1^{1/2}Q$, $\tilde{S} = Q^{-1}\Sigma_1^{1/2}V_1^T$ for an unknown 2×2 matrix Q .

Orthonormality [N2, s.34–35]. Write $\mathbf{i}_f = (\hat{a}_f^T Q, \alpha_f)$ and $\mathbf{j}_f = (\hat{b}_f^T Q, \beta_f)$, where $\hat{a}_f^T, \hat{b}_f^T$ are rows f and $F+f$ of $U_1\Sigma_1^{1/2}$. The out-of-plane components α_f, β_f multiply $z = 0$, so they are invisible in W , but $\mathbf{i}_f, \mathbf{j}_f$ must still be orthonormal:

$$\hat{a}_f^T Q Q^T \hat{a}_f + \alpha_f^2 = 1, \quad \hat{b}_f^T Q Q^T \hat{b}_f + \beta_f^2 = 1, \quad \hat{a}_f^T Q Q^T \hat{b}_f + \alpha_f \beta_f = 0, \quad f = 1..F.$$

These are $3F = 12$ quadratic equations in the 4 entries of Q and the 8 values α_f, β_f . As on [N2, s.35], they are solved with Newton’s method (Gauss–Newton, 40 deterministic starts, best least-squares solution kept). Q is defined only up to an in-plane rotation and a mirror image. I fix the rotation by putting the first edge on $+x$, and the mirror by requiring that every camera sees the front of the plane, $(\mathbf{i}_f \times \mathbf{j}_f)_z > 0$.

Scale and boundary. Orthography gives shape up to scale. With normalized coordinates $u \approx X/\bar{Z}$, $\tilde{S}$ is in units of $1/\bar{Z}$. One known length fixes it: the top edge (corner 1 to 9) is $8 \times 24.9 = 199.2$ mm. The boundary is the polygon through the outer corners of $\tilde{S}$.

2.2 Calculations

W is 8×54 . Its columns for corners 1, 9, 46 and 54 (the four outer corners) are

$$W_{[:, 1, 9, 46, 54]} = \begin{bmatrix} -0.1011 & 0.1062 & -0.0973 & 0.0926 \\ -0.0893 & 0.0792 & -0.0792 & 0.0824 \\ -0.0756 & 0.0818 & -0.0715 & 0.0750 \\ -0.0834 & 0.0836 & -0.0808 & 0.0738 \\ -0.0592 & -0.0585 & 0.0555 & 0.0567 \\ -0.0472 & -0.0722 & 0.0747 & 0.0433 \\ -0.0794 & -0.0208 & 0.0175 & 0.0828 \\ -0.0247 & -0.0730 & 0.0735 & 0.0223 \end{bmatrix},$$

(rows $\tilde{u}_{1..4}, \tilde{v}_{1..4}$; image centroids e.g. $(\bar{u}_1, \bar{v}_1) = (0.0545, -0.2421)$).

$$\sigma(W) = (0.8304, 0.5425, 0.0159, 0.0131, 0.0029, 0.0005, \dots).$$

Two singular values dominate, about $35\times$ larger than the rest, confirming rank 2 for a plane; the small remainder is perspective and corner noise. Then

$$U_1 \Sigma_1^{1/2} = \begin{bmatrix} -0.5165 & -0.0072 \\ -0.4282 & -0.0422 \\ -0.3947 & -0.0150 \\ -0.4182 & -0.0061 \\ 0.0092 & -0.3915 \\ 0.0856 & -0.4014 \\ -0.1512 & -0.3484 \\ 0.1391 & -0.3235 \end{bmatrix}, \quad Q = \begin{bmatrix} -2.0015 & 0.0446 \\ -0.0202 & -2.4469 \end{bmatrix}, \quad QQ^T = \begin{bmatrix} 4.0082 & -0.0688 \\ -0.0688 & 5.9875 \end{bmatrix}.$$

The recovered camera axes $\mathbf{i}_f = (\tilde{\mathbf{i}}_f, \alpha_f)$, $\mathbf{j}_f = (\tilde{\mathbf{j}}_f, \beta_f)$, $\mathbf{k}_f = \mathbf{i}_f \times \mathbf{j}_f$, and tilt = $\arccos(k_{f,z}/\|\mathbf{k}_f\|)$:

View	$\mathbf{i}_f$	$\mathbf{j}_f$	$\mathbf{k}_f$	Recovered tilt	Calibration tilt
1	(1.034, -0.005, 0.054)	(-0.011, 0.958, -0.300)	(-0.050, 0.310, 0.991)	17.6°	26.0°
2	(0.858, 0.084, 0.508)	(-0.163, 0.986, 0.104)	(-0.492, -0.172, 0.860)	31.2°	31.1°
3	(0.790, 0.019, 0.612)	(0.310, 0.846, -0.432)	(-0.525, 0.531, 0.662)	48.4°	47.5°
4	(0.837, -0.004, 0.529)	(-0.272, 0.798, 0.517)	(-0.424, -0.576, 0.667)	47.0°	45.5°

Check for view 2: $|\mathbf{i}_2|^2 = 0.858^2 + 0.084^2 + 0.508^2 = 1.001$ and $\mathbf{i}_2 \cdot \mathbf{j}_2 = -0.140 + 0.083 + 0.053 = -0.004 \approx 0$. The largest equation residual is 0.07 (view 1, $|\mathbf{i}_1| = 1.034$). View 1, the least tilted, has the smallest out-of-plane components, so its tilt is the least well determined (off by 8°). For a plane only $\alpha_f \beta_f$ is fixed, so each camera's tilt angle is recovered but not which way it leans. The camera positions C_f were removed by the centering trick, so orthographic factorization recovers orientations, not positions [N2, s.14].

Structure. Corner 1 is $\tilde{S}_1 = (-0.0972, -0.0624)$ and corner 9 is $(0.1008, -0.0588)$, 0.1980 apart, so the scale is $199.2/0.1980 = 1006$ mm per unit. Because normalized coordinates are $\approx X/\bar{Z}$, this scale should equal the mean camera distance, and it does: 1006 mm vs. 1057 mm from calibration. Corner 1 is then $(-97.7, -62.8)$ mm. Reprojection check, corner 1 in view 2: $\tilde{\mathbf{i}}_2^T \tilde{S}_1 = 0.8579(-0.0972) + 0.0841(-0.0624) = -0.0886$ vs. the measured $\tilde{u}_{2,1} = -0.0893$: $0.0007 \times 3337 \approx 2.3$ px.

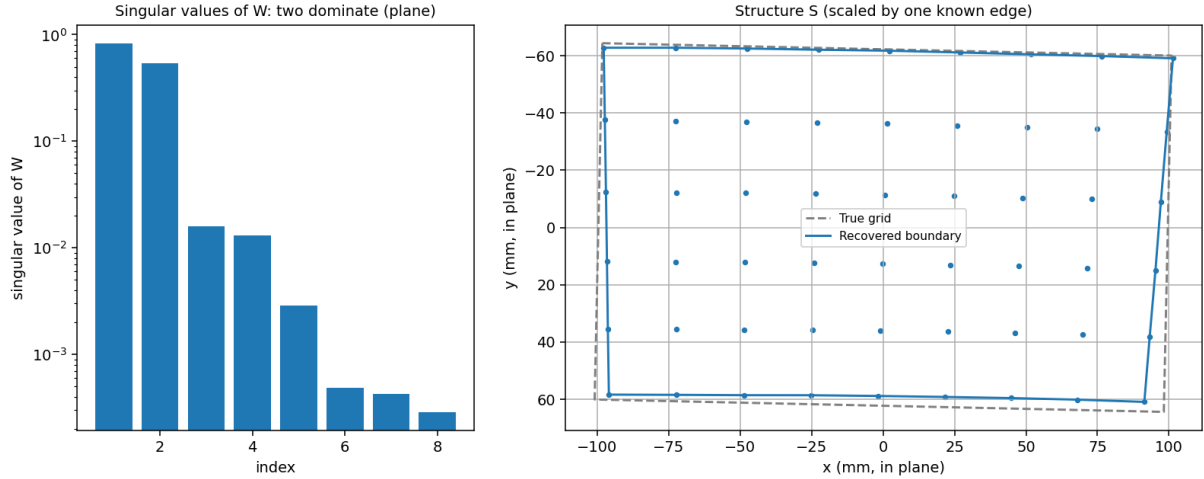


Figure 4: Left: singular values of W . Right: recovered structure and boundary (blue), with the true grid rigidly aligned (grey dashed).

Result. The recovered boundary sides are 199.2 (scale reference), 120.5, 187.4 and 121.2 mm, against the true 199.2 and 124.5 mm. All 54 points are within 7.7 mm of the true grid after rigid alignment (RMS 2.9 mm). Mean reprojection error is 2.0–6.4 px per view. Three of four camera tilts agree with the calibration to within 1.5°.

Why the errors are not zero. The orthographic model needs the depth variation to be small compared with the distance [N2, s.8]. Here the tilted board spans 8–16% of the viewing distance, so perspective foreshortens the far

edge differently in each view. The model cannot represent that, which is why the bottom edge comes out 187 mm instead of 199 mm. The four photos are also among those used for the Module 2 calibration, so K is fitted to them. Larger distances (with zoom) or the perspective extensions mentioned in [N2, s.38] would reduce the error. The app accepts any four new photos with corners clicked by hand.

References

- [N1] S. K. Nayar, *Optical Flow*, Monograph FPCV-4-3, First Principles of Computer Vision, Columbia University, April 2025.
- [N2] S. K. Nayar, *Structure from Motion*, Monograph FPCV-4-4, First Principles of Computer Vision, Columbia University, April 2025.
- [N3] S. K. Nayar, *Object Tracking*, Monograph FPCV-5-1, First Principles of Computer Vision, Columbia University, April 2025.
- B. D. Lucas and T. Kanade, “An iterative image registration technique with an application to stereo vision,” *Imaging Understanding Workshop*, 1981.
- B. K. P. Horn and B. G. Schunck, “Determining optical flow,” *Artificial Intelligence*, 1981.
- J.-Y. Bouguet, “Pyramidal implementation of the Lucas Kanade feature tracker,” Intel Corp., 2000 (used via OpenCV).
- C. Tomasi and T. Kanade, “Shape and motion from image streams under orthography: a factorization method,” *IJCV*, 1992.
- Z. Zhang, “A flexible new technique for camera calibration,” *IEEE TPAMI*, 22(11), 2000 (Module 2 calibration).
- [V1] “Penguin Walk – Zoo Basel [HD],” <https://www.youtube.com/watch?v=fdk7X4LrgS4>, sample 00:30–01:00.
- [V2] “SAND MARBLE RACE, S7: Round 1,” Jelle’s Marble Runs, <https://www.youtube.com/watch?v=0sHu99vdz-A>, sample 02:00–02:30.